Homework 2

In this assignment, as with all other work in our class, you are required to work in teams.

The primary goal of this assignment is to demonstrate two key ideas for this class:

1. You can structure a service around slow model inference so that it stays responsive when several users call it at once, and package it so that it runs the same on a teammate's laptop and on your team's cloud instance.

2. You can decide where a model should live, defend the decision with a measurement, and write the decision down where the next person will find it.

In Homework 1, your team built the installable foundation of your project, and the model behind it did not have to do anything interesting. In this assignment, the model is still a black box, but the software around it grows up: the service stops freezing while the model thinks, becomes a container image that starts with one command on your Jetstream2 instance, and the model behind it gets a deliberate home.

As before, the assignment is organized through broadly-defined Task Lists. It covers our lectures on async services, on containerization, and on serving models, one problem per week, and it is meant to be done in those weeks, not the night before it is due.

What you are building

You are continuing the project your team started in Homework 1, in the same repository. By the end of this assignment, your service should:

- answer a real request with a model call that does not block the event loop, with a measurement to prove it;
- build into a container image from your repository, with configuration supplied at run time and never baked in;
- run on your team's Jetstream2 instance beside whatever it depends on, started by one `docker compose up -d` and back on its own after an unshelve;
- reach its model through a client whose address or name comes from configuration, so that moving the model is a configuration change;
- come with a short written decision about where the model lives and what that costs, backed by numbers you measured.

Problem 1.

Now that you're more familiar with using tools for project structuring and automation of different parts of the development process, we're going to add something new: better collaboration through Git.

Your team will be using issue templates for common processes like reporting bugs, suggesting new features, or asking questions. Using issue templates benefits maintainers because it provides a consistent format that encourages users to submit issues in a uniform manner, and this also helps users, who will know what maintainers are looking for when receiving issues. Issue templates are placed in the `.github/ISSUE_TEMPLATE` directory of your repository. There are three issue templates you can download from Canvas contained in the directory `ISSUE_TEMPLATE`. These templates are provided by the well-structured Python project ir_datasets: `bug_report.md`, `documentation.md`, and `feature_request.md`. You are free to modify these templates as you see fit; they are just a suggested starting point, and if your preferences are different, that's wonderful. You can take a look at the past issues in the ir_datasets repository to get an idea of how real people used the templates. GitHub provides more information about issue templates here.

All progress in your repository should occur via a well-defined collaborative process. A good workflow you're expected to start to follow is below. **Use this workflow**:

- **Create an issue for a single task in your project**: give your issue a concise and clear name such as `add the predict endpoint`, `fix the health check timeout in the container`, `improve the README run instructions`, and so on.

- **Open a new branch**: you and your team should not be working on the main branch. Instead, once you've created an issue, you should each be working from feature branches. This ensures the main branch stays stable and promotes better code review. Each branch addresses a specific issue. The branch should be named relative to the issue. This GitHub page describes how you can link a branch to an issue.
- **Submit all work via pull requests**: a pull request is a proposal to merge changes from one branch in your project to some other branch. Typically, this involves merging changes from a feature branch into the main branch of your project. All of your pull requests should be linked to specific issues. For examples of good pull requests, take a look at Hugging Face's DataTrove library. Many other libraries have equally useful examples. You don't need to use keywords in the titles of your pull requests, but you should link them to specific issues.
- **Have someone else on the team comment on the pull request**: as a collaborative unit, your team is expected to encourage and help each other. When a teammate submits a pull request, read through their changes and leave comments as needed. **You do not have to do this for every issue or every small task in your project. Do it at least one time with a teammate to get into the habit of checking your teammates' work.**
- **Merge changes into the main branch**: once a pull request has been reviewed and approved, merge it into the main branch. Finally, close the issue associated with the change.

Every teammate should open at least one pull request that changes application code in a meaningful way.

Problem 2.

Following our configuration and async lectures, make your service stay responsive while the model thinks.

Software Task List

- ❒ Configure your application. Add a typed configuration with Pydantic Settings: the app reads its settings from the environment, with validators that fail fast at startup on bad config values.
- ❒ Put a real model call behind your predict-style endpoint, loaded once at startup, with the input validated by a Pydantic model as in Homework 1.
- ❒ Do a blocking-call audit with a measured before and after. First, put a blocking call inside an `async def` endpoint on purpose: a synchronous model call, a synchronous HTTP client, or a `time.sleep` standing in for a slow call if your model answers too quickly to show the effect (say so). Send ten requests at once with `httpx.AsyncClient` and `asyncio.gather`, as the lab did, and record the wall-clock time and the slowest request. Then fix it, with a plain `def` handler or with `asyncio.to_thread`, and measure again. Keep the blocking version reachable under a Git tag so the instruction team can see both.
- ❒ Write a table like from the lab for your own service: for one request, each step, whether the thread is computing or waiting, on what, and for about how long. Put this in a short `discussion.md`, with the two measurements under it and a sentence on what caused the difference.

Problem 3.

Following our containerization lecture, package your service into an image and run it on your instance the way we ran the case study.

Software Task List

- ❒ Write a `Dockerfile` at the root of your repository that installs your project from `uv.lock` with `uv sync --frozen` and starts the service with uvicorn. Copy the lock file and install the dependencies before you copy your source, so that the dependency layer is cached between builds. Declare a `HEALTHCHECK` against your `/health` endpoint, and run as a non-root user; the case study does not, and you should.
- ❒ Add a `.dockerignore`. The image must not contain your `.venv`, your `.git` history, or any real environment file. Configuration comes in at run time with `--env-file` or Compose's `env_file`.
- ❒ Write a `compose.yml` that starts your service and its dependencies (a database, a vector store, a cache, or nothing, in which case say so) on your Jetstream2 instance: named volumes for the data that must survive the

container, a health condition on each dependency, and `restart: unless-stopped`. On a fresh clone of your repository, `docker compose up -d` is the whole start procedure, and `/health` answers from outside the instance.

- ❒ Report the image's size on disk, how long a rebuild takes after a one-line code change (with and without a cached dependency layer), and `docker system df` on your instance after the stack is up.

Problem 5.

The quality gates from Homework 1 must all stay in place. Your GitHub Actions workflow keeps running ruff, mypy, and pytest on every push and pull request and is green at the commit you submit; add at least one test that exercises the model client behind a stub or a fake, so the suite does not need the real model to pass, and one that shows the fixed endpoint answers a health check while a slow request is in flight. Update the README so that a grader can, from a clean machine, build the image, start the stack with Compose on an instance, and call every endpoint with copy-pasteable curl commands.

Grading

- ❒ **Your documentation in your README files matters the most: it is important to be as clear as possible on how to run your code.** The best way to ensure your code runs on another machine is by doing a fresh installation of your environment to guarantee there won't be any environment issues.
- ❒ **In this assignment, your goal is to be a good member of a team working together to share code using best practices. The instruction team is looking for good Python project structuring and CI implementation using GitHub Actions.**

Grading Guide used by the TA Team.

❏ Problem 1: The three issue templates are in .github/ISSUE_TEMPLATE; work is planned in issues with clear names, developed on branches linked to them, and merged through pull requests linked to issues; at least one pull request carries a teammate's review comments; every teammate authored at least one pull request that changed application code; merged issues are closed. (2 points)

❏ Problem 2: Students should have a service that stays responsive.

- ❏ Configuration and enforcement via pydantic. (1 point)
- ❏ A real model call behind a validated predict-style endpoint, loaded once at startup. (1 point)
- ❏ The blocking-call audit: the blocking version under a Git tag, the fixed version on main, and both ten-request measurements. (1 point)
- ❏ The table in discussion.md, with the two measurements under it and the sentence on what caused the difference. (1 point)

❏ Problem 3: Students should have a container that runs on their instance.

- ❏ Dockerfile. Installs from the lock file with the dependency layer before the source, a health check, a non-root user; the image size, the rebuild times with and without the cached layer, and docker system df are reported. (2 points)
- ❏ Nothing secret inside. A .dockerignore that keeps the environment file, the virtual environment, and Git history out; configuration supplied at run time. (1 point)
- ❏ The Compose stack starts on the team's instance from a fresh clone, with named volumes, health conditions, and a restart policy, and /health answers from outside the instance. (2 points)

❏ Problem 5: Quality gates and documentation.

- ❏ The GitHub Actions workflow is green at the submitted commit, with a test that stubs the model and a test that the health check answers during a slow request. (1 point)

❑ A README a grader can follow from a clean machine: build the image, start the stack with Compose on an instance, and call every endpoint with copy-pasteable curl commands. (1 point)